

Nodges – Architekturbericht (GPT-5 Edition)

1. Architekturziele

- Interaktive 3D-Netzwerkvisualisierung mit hoher Performance und Erweiterbarkeit.
- Lose Kopplung via Ereignisse und Manager-Orchestrierung.
- Saubere Trennung in Core (Koordination), Utils (Dienste), Effects (Darstellung), Objects (Daten/Geometrie).

2. Schichten und Verantwortlichkeiten

- Presentation/Rendering: Three.js Szene, Materials, Meshes (InstancedMesh für Nodes, TubeGeometry für Edges), Effekte.
- Application/Core: State-, UI-, Layout-, Interaction-, CentralEvent-Manager.
- Domain/Utils: Datenformate, Import/Export, Validierung, Analyse, Suche, Auswahl, Pfadsuche, Performance, Gruppen.
- Integration: ConnectInterfaceAdapter, FileHandler, KeyboardShortcuts, GroundManager, EdgeLabelManager.

3. Haupt-Manager im Detail

- StateManager
 - Zentrale Quelle für Anwendungszustand (Selektion, Modi, Filter, Layer/Ebenen, GUI-Flags).
 - API: Getter/Setter, Events/Callbacks (z. T. über CentralEventManager gebunden).
- UIManager
 - UI-Overlays, Panels, Shortcuts (F1 Overlay via KeyboardShortcuts), Bindung an State und Interaktion.
- LayoutManager
 - Berechnet/steuert Layouts, Worker-Auslagerung (layout-worker), aktualisiert Positionen von Nodes/Edges.
- InteractionManager
 - Interpretiert Eingaben, nutzt RaycastManager, initiiert Selection/Highlighting, Dragging/Box-Select/Mehrfachauswahl.
- CentralEventManager
 - Pub/Sub/Eventbus für lose Kopplung, Koordination zwischen Managern.

4. Domain- und Rendering-Komponenten

- Objects
 - Node: logisches Knotendatenobjekt (Position, Name, Farbe, Größe, Metadata).
 - Edge: geometrische Repräsentation einer Kante als kurvige TubeGeometry, einschließlich Mehrkantenabstand, Materialfarbe, userData.
 - NodeMeshFactory: Herstellung von Meshes gemäß Node-Eigenschaften (Sphere/Cube/Cone), Materialvorgaben.
- Effects
 - GlowEffect: Emissive/Color-Intensität, selektions- und highlight-spezifische Presets.

- HighlightManager: Vermittelt Hover/Select-Effekte, ruft GlowEffect an Knoten/Kanten ab.

5. Utils und Dienste

- Import/Export/FileHandler
 - Ein-/Auslesen von JSON, Beispiel-Datasets, Speichern des aktuellen Graphen.
- FutureData*
 - Parser/Manager/Validator für ein zukünftiges Datenformat, inkl. Migrations-/Validierungslogik.
- RaycastManager
 - Mausposition → Raycaster → Schnittmengen (Edges vorrangig, dann InstancedMesh Nodes), throttled (50ms), lastIntersected Cache.
- SearchManager/SelectionManager/BatchOperations
 - Suchen nach Knoten/Kanten, Auswahlverwaltung (einzeln/multi), Massenoperationen, Undo-/Redo-Basis.
- LayoutGUI/NetworkAnalyzer/PathFinder
 - UI-Layoutsteuerung, Metriken/Analysen, Pfadsuche (kürzeste Wege o. ä.).
- PerformanceOptimizer
 - Optimierungen: Throttling, Batch-Updates, Sichtbarkeitsregeln, ggf. LOD-Konzepte.
- NodeGroupManager/NeighborhoodHighlighter
 - Gruppieren, Nachbarschaftsbetonung/Erweiterung.
- EdgeLabelManager/GroundManager/KeyboardShortcuts/ConnectInterfaceAdapter
 - Edge-Labels, Boden, Hilfe/Shortcuts, Integrationsadapter.

6. Datenfluss & Lebenszyklen

- Importfluss: FileHandler → ImportManager → FutureDataParser/Validator → FutureDataManager → Node/Edge-Erzeugung/Scene Aufbau.
- Interaktion: InteractionManager → RaycastManager → SelectionManager → StateManager → HighlightManager/GlowEffect → UIManager.
- Layoutzyklus: LayoutManager (ggf. Worker) → Node/Edge Positionen → Render.
- Export: ExportManager → FileHandler.

7. Ereignisse und Zustandsübergänge

- Wichtige Events: selectionChanged, layoutUpdated, dataImported, hoverChanged, nodeDragged, filterChanged.
- Zustandsautomat (Auswahl): Idle → Hover → Selected → MultiSelecting → Selected/Idle; Escape löscht Auswahl.

8. Nebenläufigkeit/Worker

- layout-worker.js: Auslagerung kostenintensiver Layoutberechnungen; Main-Thread empfängt Ergebnis und aktualisiert Szene.

9. Skalierung und Performance

- Instanced Meshes für Nodes; Kanten als einfache TubeGeometry mit begrenzten Segmenten (z. B. 5) zur Kostenkontrolle.

- Raycast-Throttling und Caching.
- Batch-Operationen für Massenänderungen.
- Potenzial: Spatial Indexing, Visibility Culling, GPU-Layouts, WebGPU-Pfade künftig.

10. Fehlertoleranz & Validierung

- FutureDataValidator und Parser zur Sicherstellung korrekter Daten.
- Fallback-Farben, Default-Geometrien (Node/Edge) minimieren Fehlerfolgen.

11. Erweiterungspunkte

- Neue Layouts im LayoutManager + Worker.
- Neue Effekt-Plugins im Effects-Layer.
- Zusätzliche Parser/Exporter im Utils-Layer.
- Zusätzliche UI-Panels/Shortcuts im UIManager/KeyboardShortcuts.

12. Diagramme (extern)

Die Diagramme sind ausgelagert in den Dateien unter `diagrams_gpt5/` und werden extern gerendert. Beim Rendern ist sicherzustellen, dass die Schriftgröße auf A4 nicht unter 11pt liegt. In den .mmd-Dateien ist eine Mermaid-Init-Direktive mit `fontSize: '15px'` hinterlegt, um dies sicherzustellen.

Siehe nummerierte Referenzen [3.1]–[3.9].

13. Referenzen (nummeriert)

- [1] bericht_gpt5.md – Projektbericht
- [2] bericht_architektur_gpt5.md – Dieses Dokument
- [3.1] diagrams_gpt5/architecture_overview.mmd – Architekturübersicht
- [3.2] diagrams_gpt5/module_dependency.mmd – Modulabhängigkeiten
- [3.3] diagrams_gpt5/data_flow.mmd – Datenfluss
- [3.4] diagrams_gpt5/event_sequence_selection.mmd – Sequenzdiagramm: Selection
- [3.5] diagrams_gpt5/import_pipeline.mmd – Flowchart: Import-Pipeline
- [3.6] diagrams_gpt5/state_machine_selection.mmd – Zustandsautomat: Selection
- [3.7] diagrams_gpt5/mindmap_system.mmd – Mindmap: System Overview
- [3.8] diagrams_gpt5/flowchart_startup.mmd – Flowchart: App-Startup
- [3.9] diagrams_gpt5/class_map_core.mmd – Klassendiagramm: Core & Effects

Ende des Architekturberichts.